P3 单周期 CPU 设计文档

目录

- 一、CPU 设计方案综述
- 1.1 总体设计
- 1.2 顶层电路
- 1.3 顶层观察信号
- 1.4 设计草稿与搭建顺序
- 二、指令格式与 RTL
- 2.1 指令字段
- 2.2 RTL 记号
- 2.3 已实现指令
- 三、关键模块设计
- 3.1 PC：程序计数器
- 3.2 IM：指令存储器
- 3.3 SPLITTER：指令字段拆分
- 3.4 GRF：通用寄存器堆
- 3.5 EXT：立即数扩展
- 3.6 ALU：算术逻辑单元
- 3.7 DM：数据存储器
- 3.8 BranchControl：条件分支判断
- 3.9 NPC：下一指令地址计算
- 3.10 PCPlus4：链接写回辅助模块
- 四、Controller 设计
- 4.1 指令识别与编号
- 4.2 MUX 查表结构
- 4.3 控制信号说明
- 4.4 当前指令控制表
- 五、典型指令的数据通路
- 5.1 寄存器运算：add/sub、addu/subu
- 5.2 立即数运算：ori/addiu/xori/lui
- 5.3 访存：lw/sw
- 5.4 条件分支：beq/bne
- 5.5 跳转与链接：j/jal/jr/jalr
- 六、测试方案与结果
- 6.1 测试目标与方法

- 6.2 定向测试覆盖方案
- 6.3 当前版本的实际测试结果
- 七、历史问题修复记录与设计注意事项
- 八、扩展指令时的 RTL 判断表
- 九、思考题
- 9.1 哪些模块存储状态，哪些模块实现状态转移?
- 9.2 IM用ROM、DM用RAM、GRF用Register是否合理?
- 9.3 是否额外设计了模块？设计思路是什么?
- 9.4 为什么nop可以不加入控制信号真值表?
- 9.5 如何评价MIPS单元6.1给出的测试样例?
- 十、参考资料与版本说明

一、CPU 设计方案综述

1.1 总体设计

本 CPU 使用 Logisim 2.7.1 搭建，采用 32 位、单周期、指令存储器与数据存储器分离的结构。每条指令在一个时钟周期内完成取指、译码、执行、访存和写回；PC、GRF 和 DM 在时钟上升沿更新相应状态，其他主要模块采用组合逻辑。

当前完成的指令为：

```
add、sub、addu、subu、ori、lui、addiu、xori、lw、sw、beq、bne、j、jal、jr、jalr、nop
```

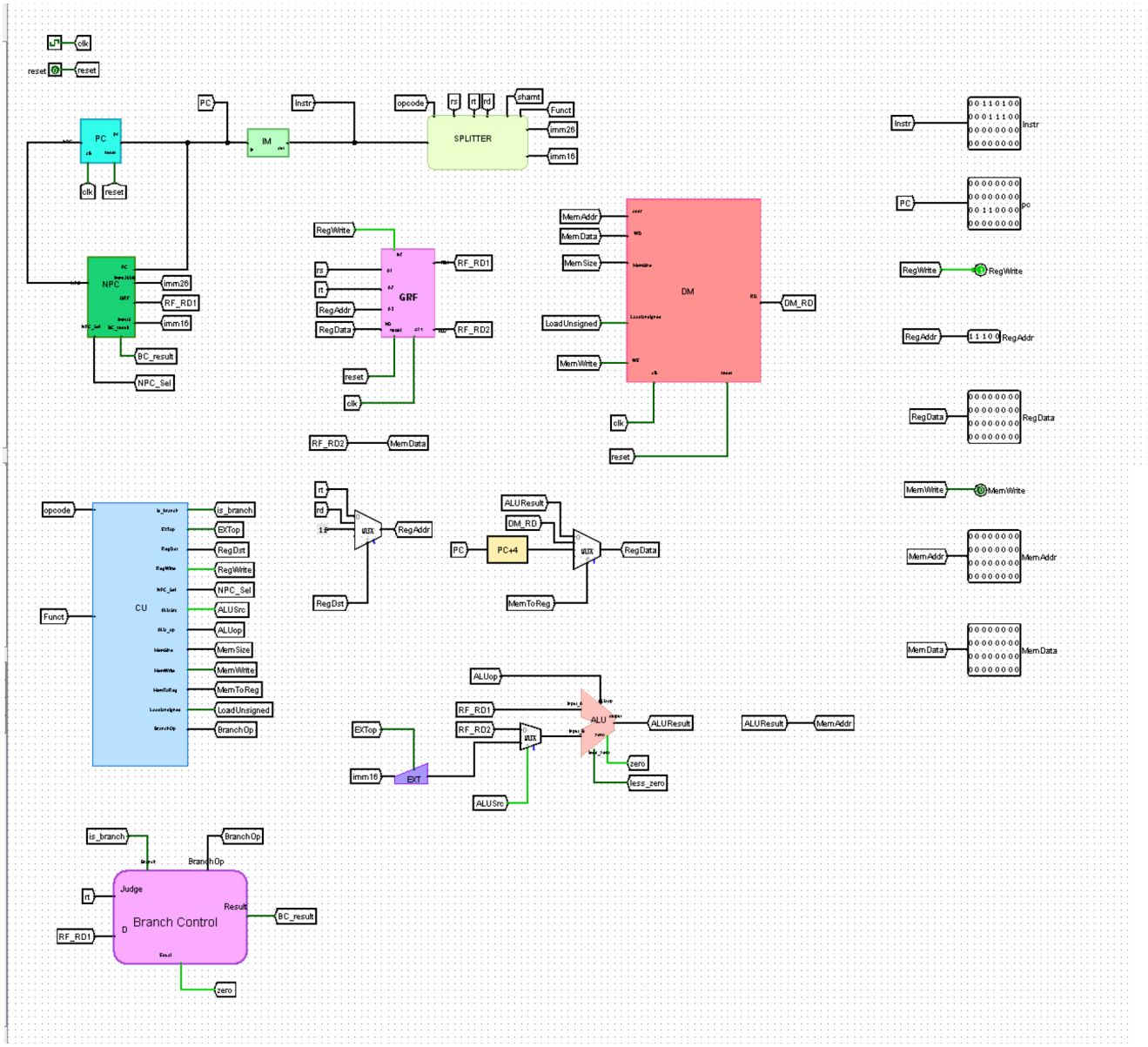
课程要求的基本指令为 `add`、`sub`、`ori`、`lw`、`sw`、`beq`、`lui`、`nop`，其余为扩展指令。题目约定 `add/sub` 按无符号加减处理，不考虑溢出；本设计保留运算结果的低32位。

其中 `nop` 为全零机器码，通过默认控制值实现，不单独占用有效指令编号。

DM 已具备字节和半字访问能力；BranchControl 已具备与零比较的分支判断能力。这些是部件扩展能力，**不代表当前 CU 已经识别 lb、lh、bltz 等指令。**

项目	本设计的约定
指令长度、主要数据通路宽度	32 位
通用寄存器	32 个，编号0～31，每个32位
`$zero`	读取恒为0，写入无效
初始 PC	`0x00003000`
复位	高电平有效，PC、GRF、DM异步清零其内部存储状态；PC对外显示初始地址
数据存储顺序	小端序
普通指令下一地址	`PC+4`
链接返回地址	`PC+4`，无延迟槽
溢出处理	运算保留低32位，不实现溢出异常

1.2 顶层电路



图中采用同名 Tunnel 连接分散的数据和控制信号。顶层由 PC、NPC、IM、SPLITTER、Controller、GRF、EXT、ALU、DM、BranchControl 和 PCPlus4 组成。

模块	作用
PC	保存当前指令地址
NPC	计算并选择下一条指令地址
IM	根据 PC 读取机器指令，是工程中唯一的 ROM
SPLITTER	提取指令字段
Controller	识别指令，通过 MUX 查表输出控制信号
GRF	提供两个组合读端口、一个时钟写端口
EXT	对16位立即数零扩展或符号扩展
ALU	完成算术、逻辑运算，并独立比较两个输入
DM	保存数据，内部由四块8位RAM组成
BranchControl	根据分支种类和寄存器值判断是否跳转
PCPlus4	为链接指令提供写回的 `PC+4`

1.3 顶层观察信号

信号	位宽	信号来源
Instr	32	IM输出的当前指令
pc	32	PC寄存器的对外输出，内部Tunnel仍命名为PC
RegWrite	1	GRF实际使用的写使能
RegAddr	5	写地址MUX的输出，接GRF.A3
RegData	32	写回MUX的输出，接GRF.WD
MemWrite	1	DM实际使用的写使能
MemAddr	32	ALU计算出的完整字节地址
MemData	32	GRF.RD2，即写入内存的原始数据

`MemAddr` 不使用截位后的 RAM 字地址。`MemWrite=0` 时，地址和数据仍可能有数值，它们不表示正在写内存。

1.4 设计草稿与搭建顺序

先从指令 RTL 提取“读什么、算什么、写哪里、下一条去哪里”，再决定模块接口和 MUX 选择。总体数据流如下，带存储的部件为 PC、GRF、DM，IM保存固定程序。

```
PC → IM → 指令字段拆分 → Controller → 各模块控制端
          |
          ├─ rs、rt → GRF ─ RD1 ──────────┐
          |            └ RD2 → ALUSrc MUX ├→ ALU → DM地址
          └─ imm16 → EXT ──────────┘          └→ 运算结果

GRF.RD2 ──────────────────────────────────────→ DM写数据
ALU结果 / DM读数据 / PC+4 → 写回MUX → GRF
PC、imm16、imm26、GRF.RD1、分支结果 → NPC → PC
```

搭建时先接通 ori/lui 的取指、译码和寄存器写回，再加入 add/sub，随后通过 sw/lw 接通数据存储器，最后加入 beq 和其他跳转指令。每加入一类指令，既检查目标指令，也回归顺序执行、写使能及已有指令，防止新接口改变原有行为。

顶层唯一外部输入为高电平有效的异步复位 reset，clk 使用内置 Clock。复位用于建立初始状态，正常运行时每个上升沿完成一次状态更新。

二、指令格式与 RTL

2.1 指令字段

类型	31:26	25:21	20:16	15:11	10:6	5:0
R型	opcode	rs	rt	rd	shamt	funct
I型	opcode	rs	rt	imm16的一部分	imm16的一部分	imm16的一部分
J型	opcode	instr_index的一部分	instr_index的一部分	instr_index的一部分	instr_index的一部分	instr_index的一部分

I型的 imm16=Instr[15:0]；J型的 instr_index=Instr[25:0]。

2.2 RTL 记号

- R[x]：GRF中编号为x的寄存器。
- SE(x)：将x符号扩展到32位；ZE(x)：将x零扩展到32位。
- M32[a]：字节地址a处的32位字，本设计的lw/sw要求地址按4字节对齐。
- PC4=PC+4；BTarget=PC4+(SE(imm16)<<2)。
- JTarget={PC4[31:28], Instr[25:0], 2'b00}。
- 所有32位结果按低32位保存。没有特殊说明时，下一地址为PC4。

2.3 已实现指令

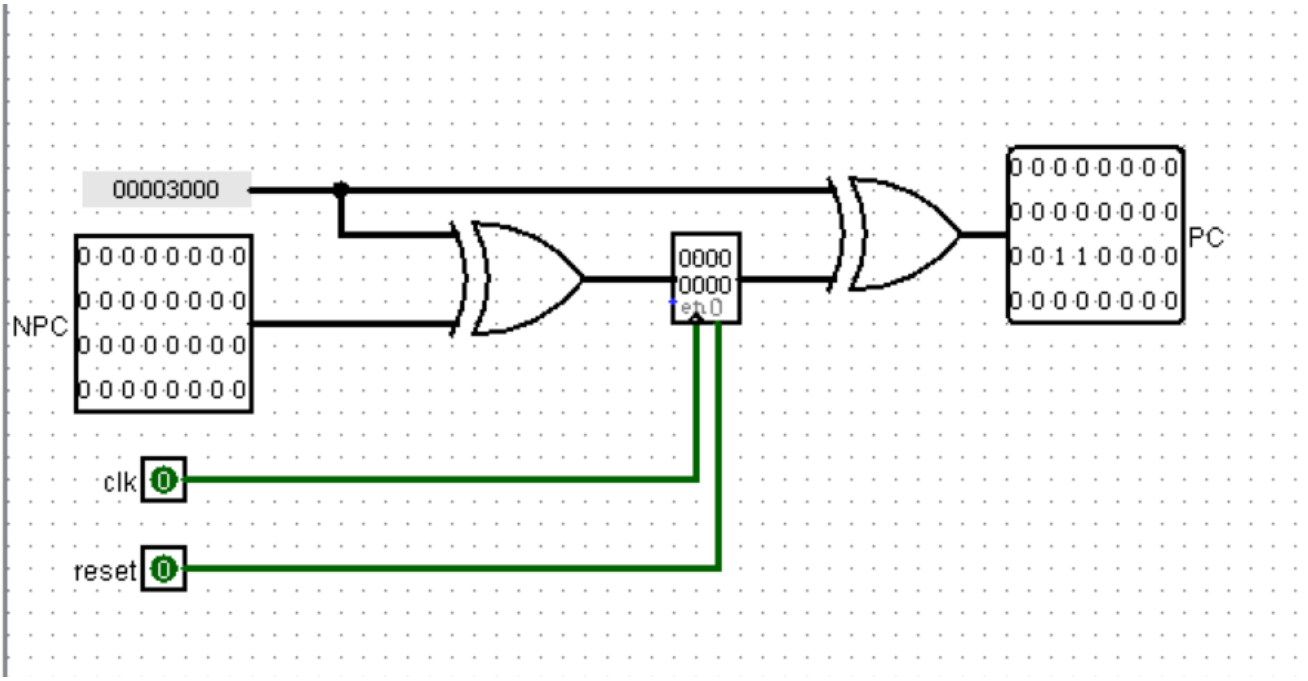
下表的 opcode、funct 用十六进制表示；它们是指令编码，与 CU 内部编号不同。

指令	格式	opcode	funct	RTL
add rd,rs,rt	R	00	20	`R[rd] ← R[rs]+R[rt]`
sub rd,rs,rt	R	00	22	`R[rd] ← R[rs]-R[rt]`
addu rd,rs,rt	R	00	21	`R[rd] ← R[rs]+R[rt]`
subu rd,rs,rt	R	00	23	`R[rd] ← R[rs]-R[rt]`
ori rt,rs,imm	I	0D	—	`R[rt] ← R[rs] OR ZE(imm16)`
lui rt,imm	I	0F	—	`R[rt] ← {imm16,16'b0}`
addiu rt,rs,imm	I	09	—	`R[rt] ← R[rs]+SE(imm16)`
xori rt,rs,imm	I	0E	—	`R[rt] ← R[rs] XOR ZE(imm16)`
lw rt,imm(rs)	I	23	—	`R[rt] ← M32[R[rs]+SE(imm16)]`
sw rt,imm(rs)	I	2B	—	`M32[R[rs]+SE(imm16)] ← R[rt]`
beq rs,rt,imm	I	04	—	相等时`PC←BTarget`，否则`PC←PC4`
bne rs,rt,imm	I	05	—	不等时`PC←BTarget`，否则`PC←PC4`
j index	J	02	—	`PC←JTarget`
jal index	J	03	—	`R[31]←PC4`, `PC←JTarget`
jr rs	R	00	08	`PC←R[rs]`
jalr rd,rs	R	00	09	`R[rd]←PC4`, `PC←R[rs]`
nop	R型全零编码	00	00	不写GRF或DM，`PC←PC4`

`addiu` 的立即数仍需符号扩展；名称中的“u”不表示立即数零扩展。`jalr` 的跳转地址取写回之前的 `R[rs]`。

三、关键模块设计

3.1 PC：程序计数器



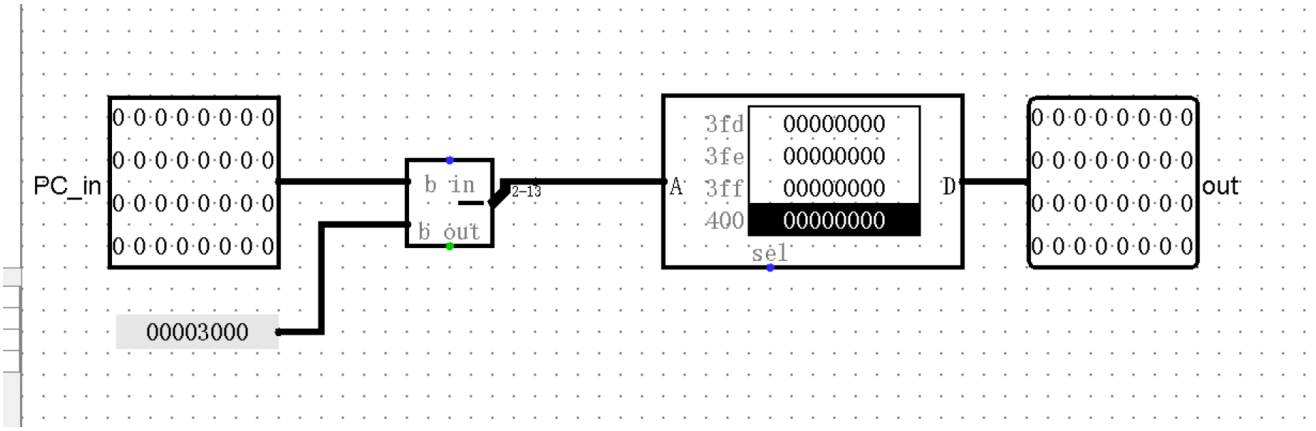
端口	方向	位宽	含义
NPC	输入	32	下一条指令地址
clk	输入	1	时钟，上升沿更新
reset	输入	1	高电平异步复位
PC	输出	32	当前指令地址

PC使用32位Register保存状态。为使清零后的寄存器对应初始地址 `0x3000`，在输入、输出两侧各放置一个与常量 `0x3000` 的异或电路。设内部寄存器为Q：

```
寄存器输入 = NPC XOR 0x00003000
对外PC     = Q   XOR 0x00003000
```

复位令Q=0，因此对外PC=0x3000。正常写入后，两次异或相互抵消，对外PC等于上升沿采样到的NPC。这里必须依靠复位使Q清零，双异或本身不负责清除未知状态。

3.2 IM：指令存储器



图中ROM输入取减去 `0x3000` 后的 `[13:2]`，对应12位字地址；ROM数据宽度为32位。

端口	方向	位宽	含义
PC_in	输入	32	当前指令字节地址
out	输出	32	当前机器指令

IM采用一块地址宽度12位、数据宽度32位的ROM，共4096个指令字。将PC减去起始地址0x3000，再取结果的 `[13:2]` 作为ROM地址：

```
ROM地址 = ((PC - 0x00003000) >> 2) 的低12位
```

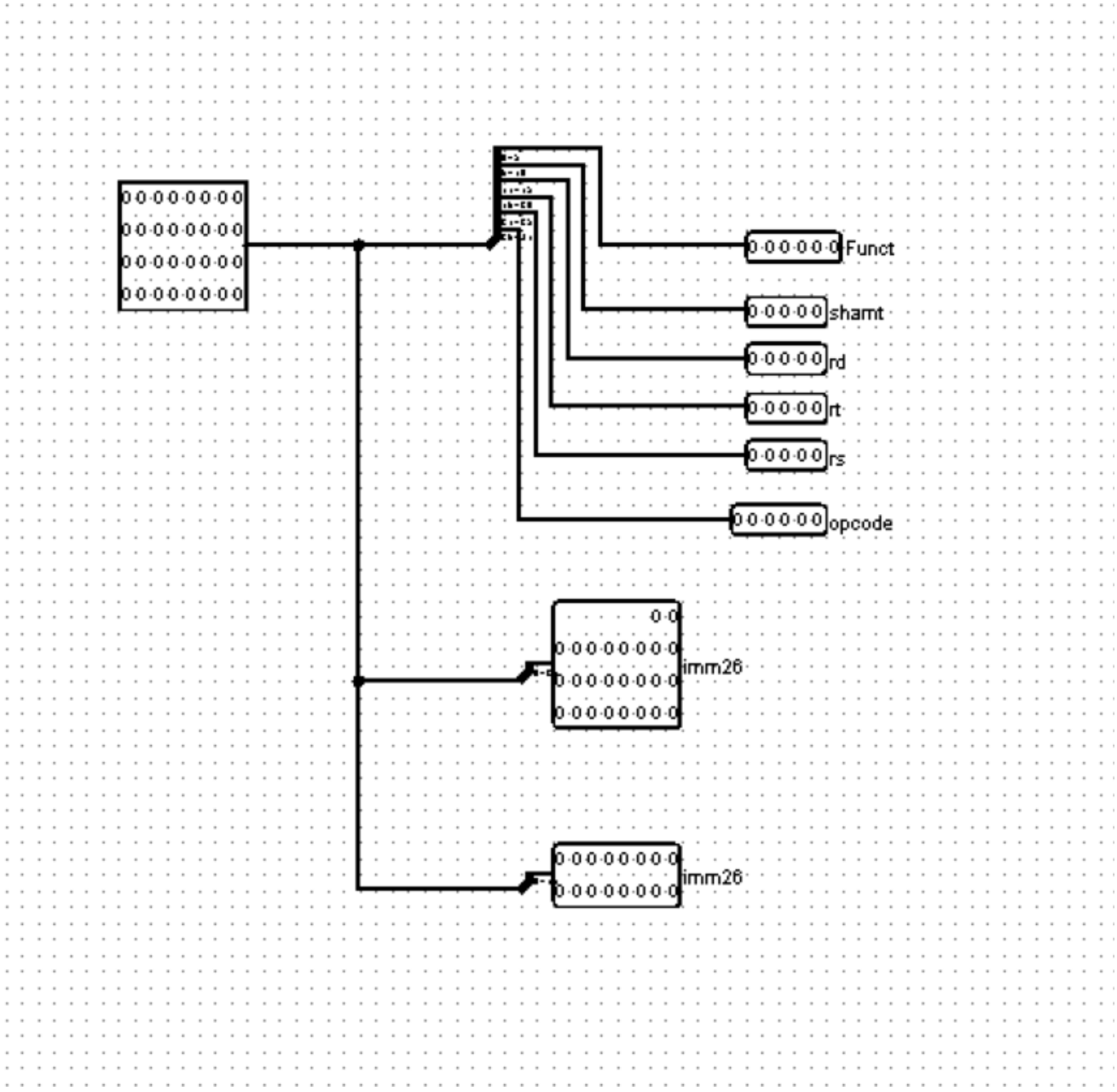
因此 `PC=0x3000` 对应ROM第0格，`PC=0x3004` 对应第1格。ROM容量相当于16 KiB，正常连续映射范围为0x00003000～0x00006FFF；电路仅截取地址位，不实现越界异常，程序应运行在约定的地址范围内。

机器码通过ROM的“Load Image”加载，常用文本格式为：

```
v2.0 raw
34081234
3c098001
```

每个十六进制数是一条32位指令。由于起始地址转换已在IM中完成，文件从第0格装入即可。

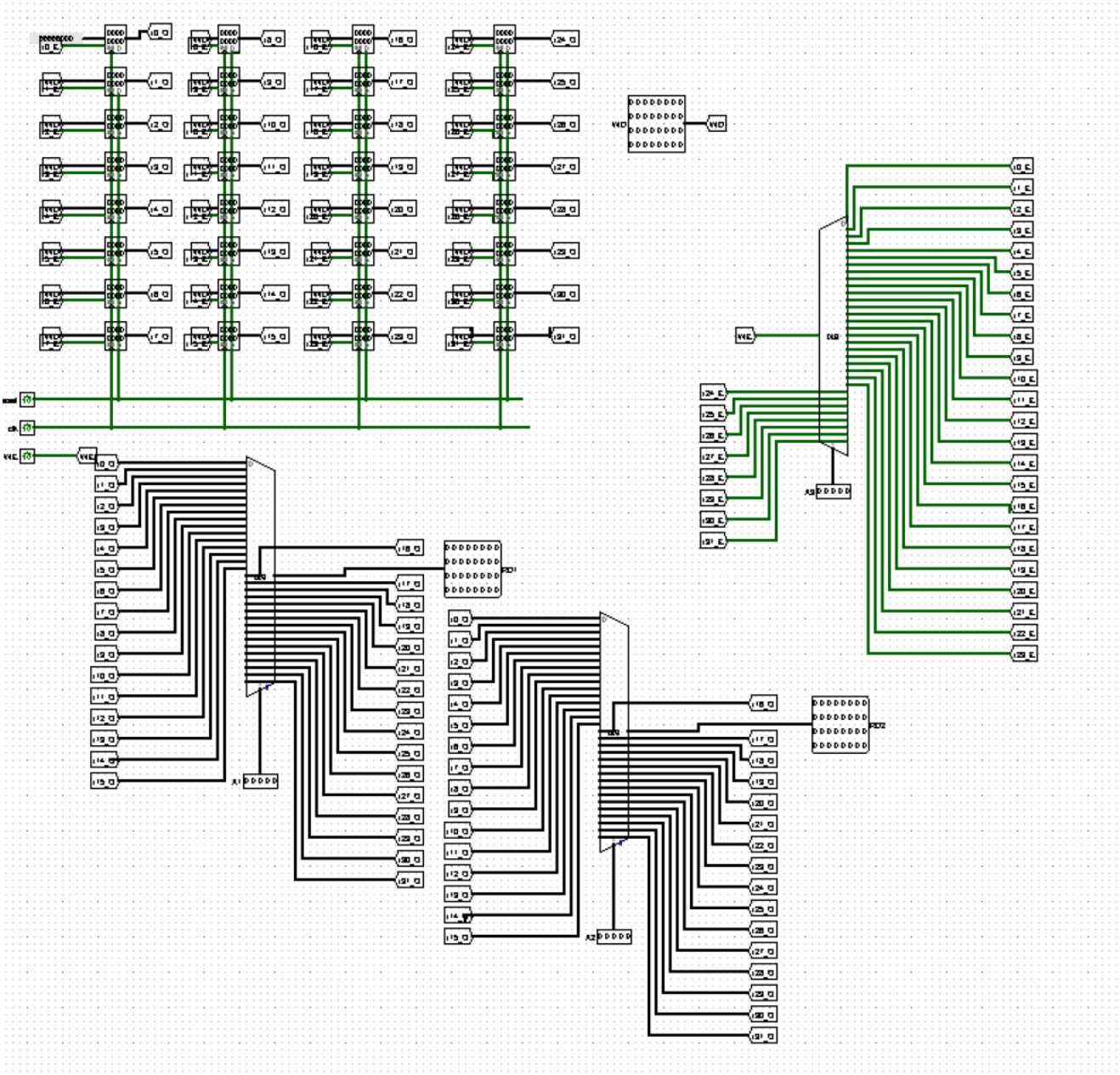
3.3 SPLITTER：指令字段拆分



字段	位宽	指令中的位置	主要用途
opcode	6	31:26	CU识别指令类别
rs	5	25:21	GRF.A1
rt	5	20:16	GRF.A2、I型写地址、BranchControl.Judge
rd	5	15:11	R型写地址
shamt	5	10:6	预留移位指令字段
Funct	6	5:0	CU识别SPECIAL类R型指令
imm16	16	15:0	EXT及分支偏移
imm26	26	25:0	J型地址拼接

截图下方16位输出的旧标签仍写作imm26，文档统一按其实际位宽和主电路用途称为imm16。

3.4 GRF：通用寄存器堆



端口	方向	位宽	含义
A1、A2	输入	各5	两个读寄存器编号
A3	输入	5	写入寄存器编号
WD	输入	32	写入数据
WE	输入	1	写使能
clk	输入	1	上升沿写入
reset	输入	1	异步清零
RD1、RD2	输出	各32	两个读端口的数据

GRF由32个32位寄存器、写地址译码电路和两个读MUX组成。WD分发到各寄存器的数据输入端；A3与WE共同决定允许哪个寄存器写入。RD1、RD2根据A1、A2组合读出，读操作不依赖WE是否为0。

0号寄存器禁止写入，其输出恒为0。其他寄存器在WE有效的上升沿写入WD。读写同一编号时，上升沿之前读到旧值，状态更新并传播后读到新值。

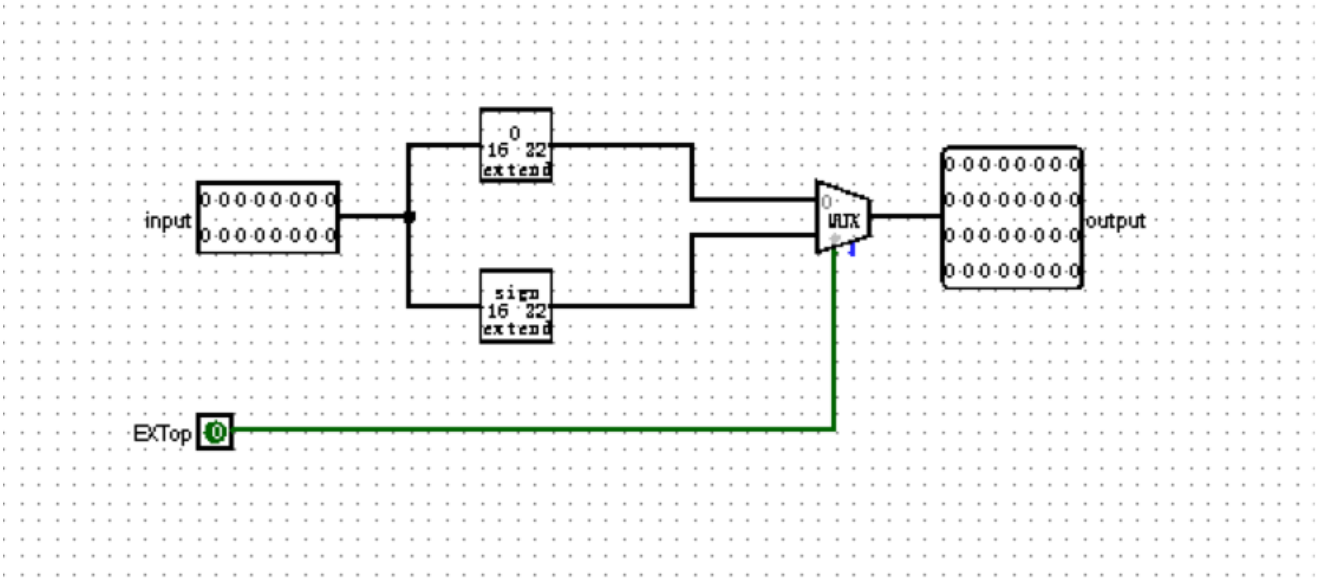
主电路的写地址MUX统一选择：

RegDst	GRF.A3 / RegAddr
00	rt
01	rd
10	常量31，用于jal
11	保留，不使用

写回数据MUX统一选择：

MemToReg	GRF.WD / RegData
00	ALUResult
01	DM.RD
10	PCPlus4输出，用于jal/jalr
11	保留，不使用

3.5 EXT：立即数扩展

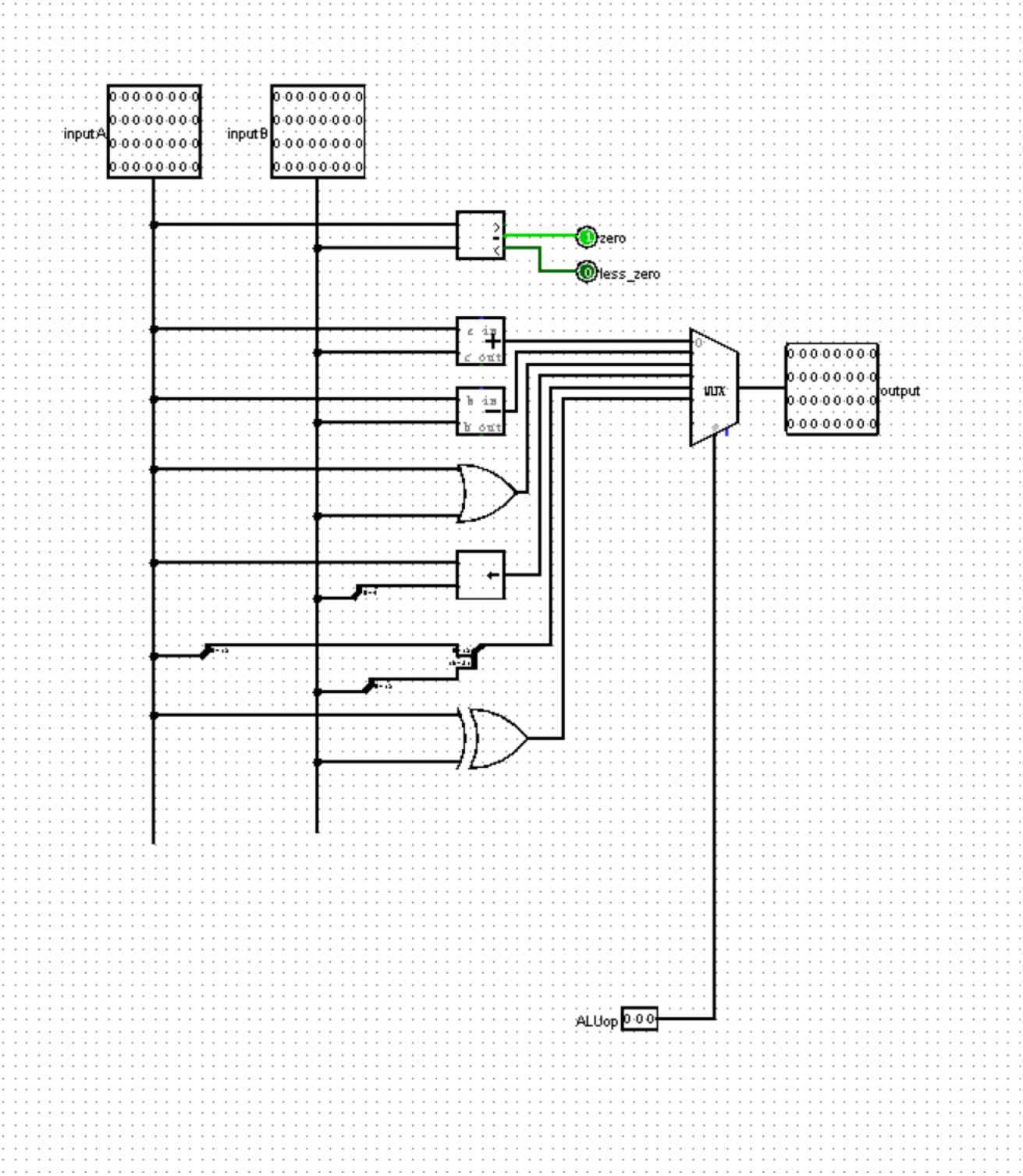


端口	方向	位宽	含义
input	输入	16	imm16
EXTop	输入	1	扩展方式
output	输出	32	扩展结果

EXTop	功能	表达式
0	零扩展	`{16'b0, imm16}`
1	符号扩展	`{{16{imm16[15]}}, imm16}`

ori/xori使用零扩展；addiu及lw/sw地址偏移使用符号扩展。分支偏移在NPC内部另行处理，不依赖主电路EXT的输出。

3.6 ALU：算术逻辑单元



端口	方向	位宽	含义
inputA	输入	32	接GRF.RD1
inputB	输入	32	由ALUSrc选择GRF.RD2或EXT输出
ALUop	输入	3	选择结果
output	输出	32	ALUResult
zero	输出	1	A是否等于B
less_zero	输出	1	A是否小于B，当前为无符号比较

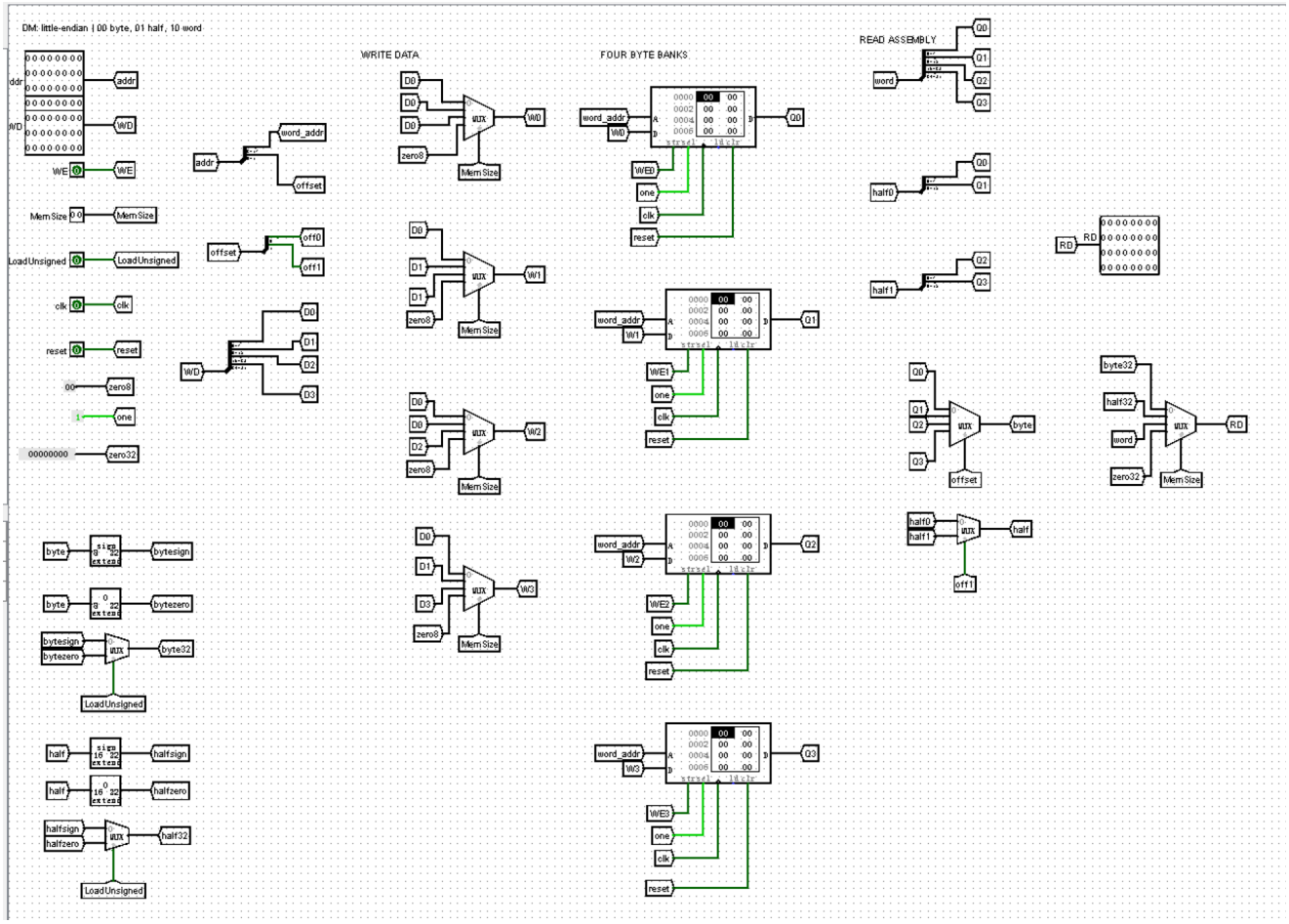
ALUop	十进制	结果
000	0	A+B，保留低32位
001	1	A-B，保留低32位
010	2	A OR B
011	3	A逻辑左移B[4:0]位
100	4	`{B[15:0], A[15:0]}`
101	5	A XOR B
110、111	6、7	未使用

lui的标准编码中rs为0，因此A来自零寄存器，拼接结果为 `{imm16,16'b0}`。

比较器独立于结果MUX：`zero`表示A==B，并不表示ALUResult==0，也不要求先选择减法。`less_zero`这一名称容易产生误解，其实际含义是无符号A<B，不能直接用于判断负数。BranchControl内部使用独立的有符号比较器。

当前ALU虽然预留左移功能，但CU尚未识别sll等移位指令，不能据此认定整机已经支持这些指令。

3.7 DM：数据存储器



端口	方向	位宽	含义
addr	输入	32	完整字节地址，接ALUResult
WD	输入	32	写数据，接GRF.RD2
WE	输入	1	写使能，接MemWrite
MemSize	输入	2	访问宽度
LoadUnsigned	输入	1	字节/半字读取的扩展方式
clk	输入	1	上升沿写入
reset	输入	1	异步清空四块RAM
RD	输出	32	读取及扩展后的数据

DM采用四块地址宽度16位、数据宽度8位的RAM，共用字地址 `addr[17:2]`，总容量256 KiB；`addr[1:0]`为字内偏移。高于bit17的地址位不参与寻址，电路不检测越界或未对齐异常。

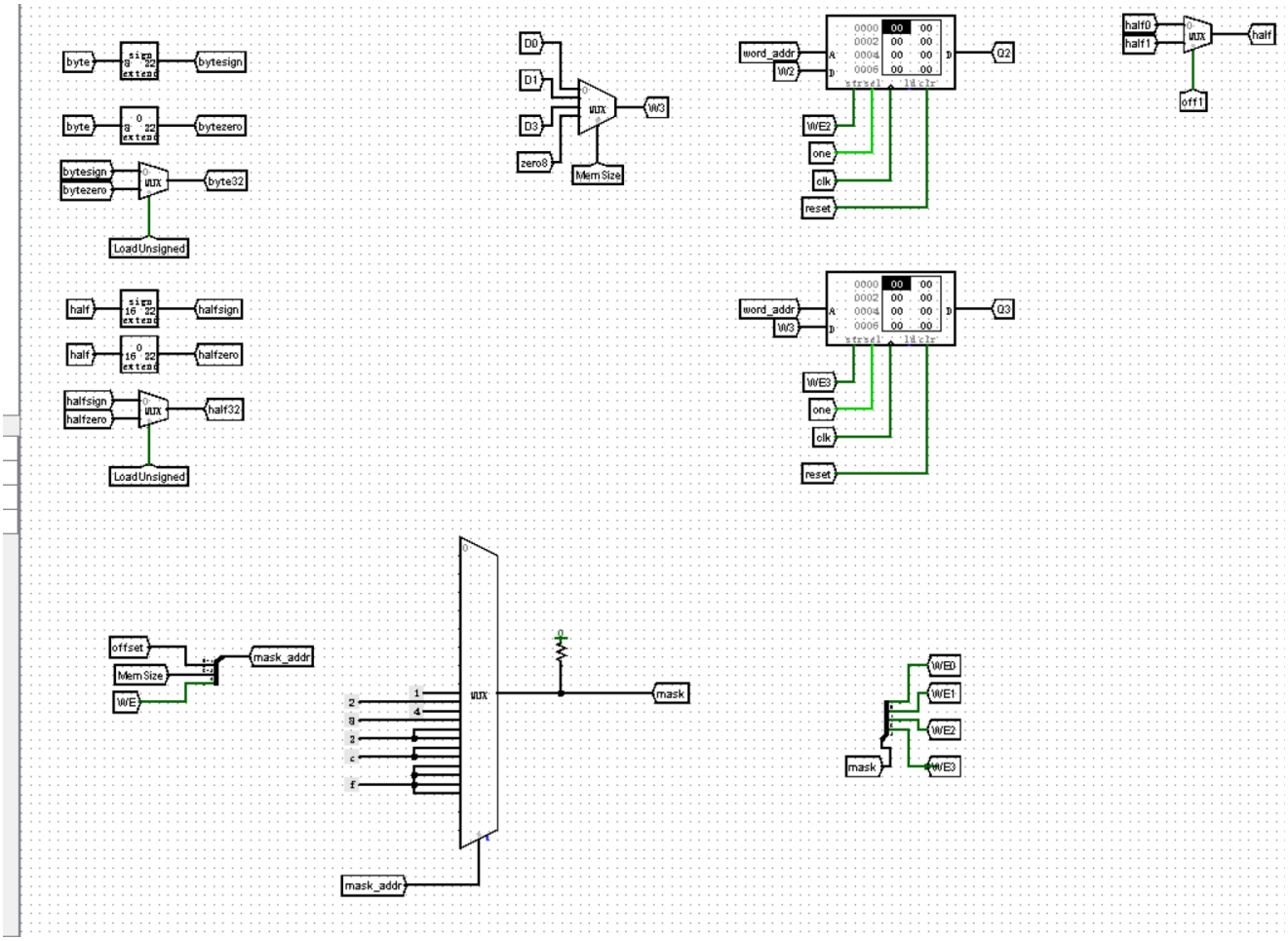
小端序对应关系如下：

RAM	字内偏移	输出对应位置
RAM0	0	word[7:0]
RAM1	1	word[15:8]
RAM2	2	word[23:16]
RAM3	3	word[31:24]

MemSize	访问宽度	读取处理
00	字节	根据addr[1:0]选择Q0～Q3，再扩展到32位
01	半字	根据addr[1]选择{Q1,Q0}或{Q3,Q2}，再扩展
10	字	直接拼接{Q3,Q2,Q1,Q0}
11	保留	读取返回0，禁止写入

字节/半字读取时，LoadUnsigned=0表示符号扩展，为1表示零扩展；整字读取不依赖该信号。半字访问要求地址为偶数，整字访问要求按4字节对齐。

写入数据与写掩码



设 `D0=WD[7:0]`、`D1=WD[15:8]`、`D2=WD[23:16]`、`D3=WD[31:24]`：

操作	RAM0输入	RAM1输入	RAM2输入	RAM3输入
字节写	D0	D0	D0	D0
半字写	D0	D1	D0	D1
整字写	D0	D1	D2	D3

四块RAM是否写入，由 `mask[3:0]` 分别决定。控制输入拼接为：

```
mask_addr = {WE, MemSize[1:0], addr[1:0]}
```

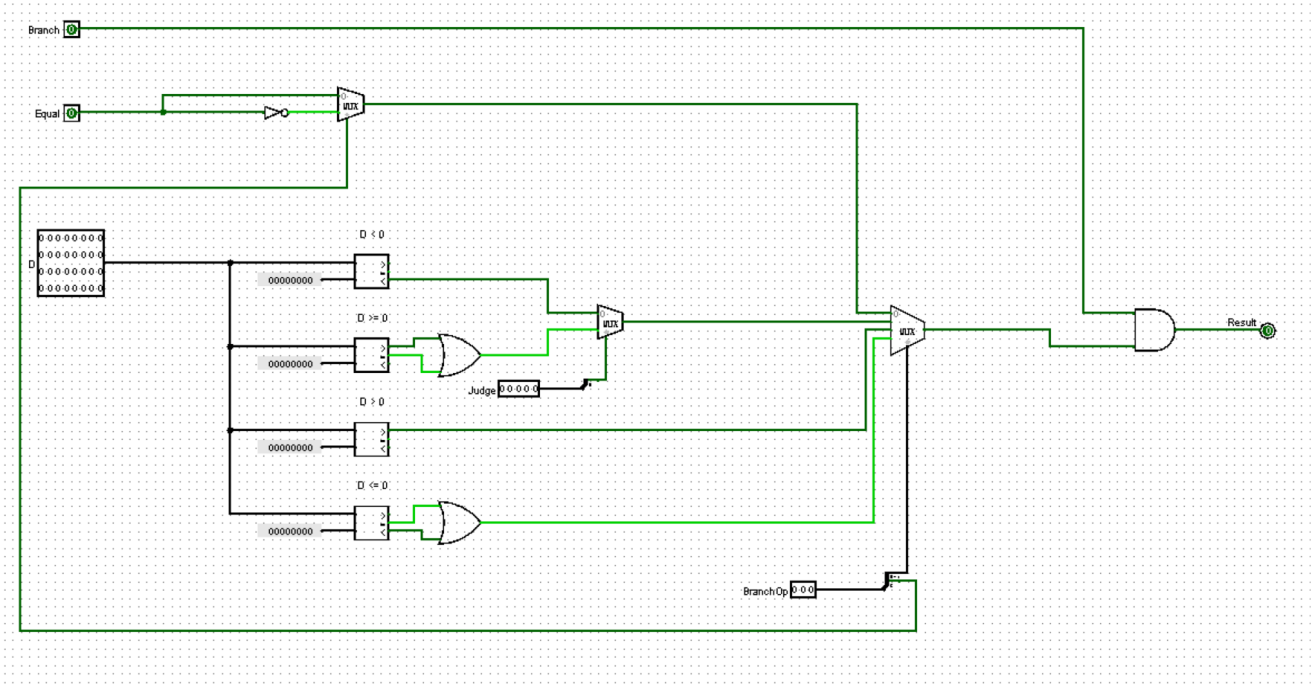
mask_addr连接32选1、数据宽度4位的MUX；MUX每个输入是常量，不使用ROM。

WE	MemSize	偏移	mask[3:0]
0	任意	任意	0000
1	00	00、01、10、11	分别为0001、0010、0100、1000
1	01	addr[1]=0	0011
1	01	addr[1]=1	1100
1	10	任意	1111
1	11	任意	0000

因此sb只改一个字节，sh只改一个半字，sw更新四个字节。其余字节不需要先读出再拼回。

当前整机CU只接入lw/sw；lb/lbu/lh/lhu/sb/sh还需要增加译码、编号和控制表项。

3.8 BranchControl：条件分支判断



端口	方向	位宽	含义
Branch	输入	1	接CU.is_branch
BranchOp	输入	3	分支条件选择
Judge	输入	5	接Instr[20:16]，区分bltz/bgez
D	输入	32	接GRF.RD1
Equal	输入	1	接ALU.zero，表示两个寄存器相等
Result	输出	1	分支是否成立，主电路中名为BC_result

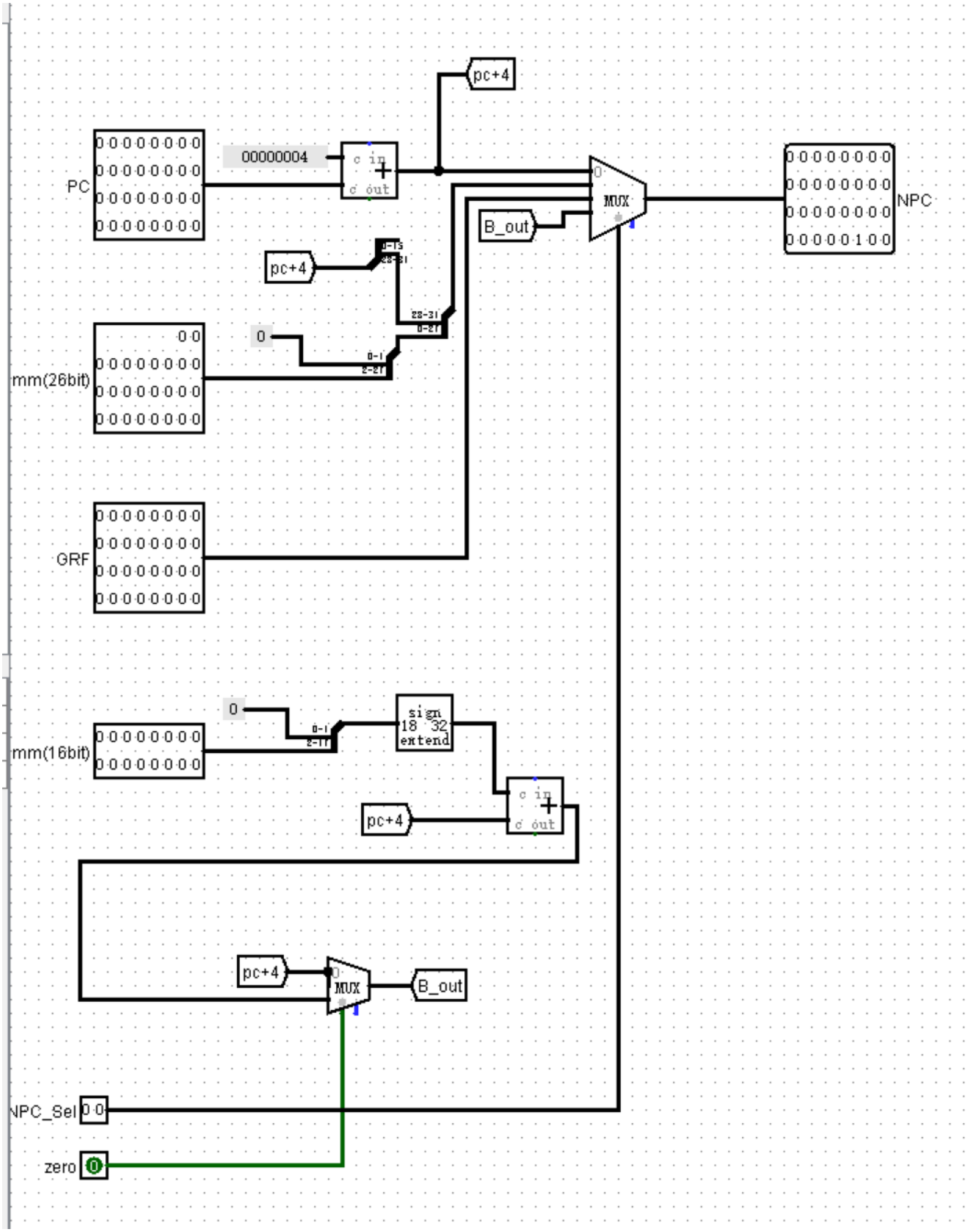
BranchOp	判断条件
000	Equal，用于beq
001	Judge=0时D<0，Judge=1时D>=0，用于bltz/bgez
010	D>0，用于bgtz
011	D<=0，用于blez
100	NOT Equal，用于bne
101 ~ 111	保留，不使用；当前实现不保证统一返回0

与零的大小比较采用有符号补码模式。内部以BranchOp低2位选择条件组，以高位切换Equal与NOT Equal。Judge只提取bit0；若以后添加REGIMM类指令，CU须限制合法rt字段，不能把该组全部编码直接视为bltz/bgez。

```
Result = Branch AND 所选条件
```

Branch=0时，结果恒为0。当前CU实际使用beq和bne；其他条件只是该模块的预留能力。两条分支的ALUSrc均为0，确保Equal比较的是GRF的两个读出值。

3.9 NPC：下一指令地址计算



端口	方向	位宽	含义
PC	输入	32	当前指令地址
imm(26bit)	输入	26	J型指令索引
imm(16bit)	输入	16	分支偏移
GRF	输入	32	接GRF.RD1，用于jr/jalr
NPC_Sel	输入	2	地址来源选择
zero	输入	1	外观标为BC_result，接BranchControl.Result
NPC	输出	32	下一指令地址

这里内部Pin沿用旧名zero，但实际输入已经是最终分支条件BC_result，而不是ALU原始相等结果。

NPC_Sel	输出
00	PC4
01	`{PC4[31:28], imm26, 2'b00}`
10	GRF.RD1
11	BC_result为1时取BTarget，否则取PC4

分支目标计算：先在imm16低位拼接两个0，得到18位数；将其符号扩展为32位，再与PC4相加。应先扩展到足够宽度再进行移位，避免把偏移高位截掉。

条件分支的二选一MUX放在地址MUX的输入3之前。这样只有NPC_Sel=3时，BC_result才影响结果，j/jr不会因为BranchControl.Result=0而被强制改回PC4。

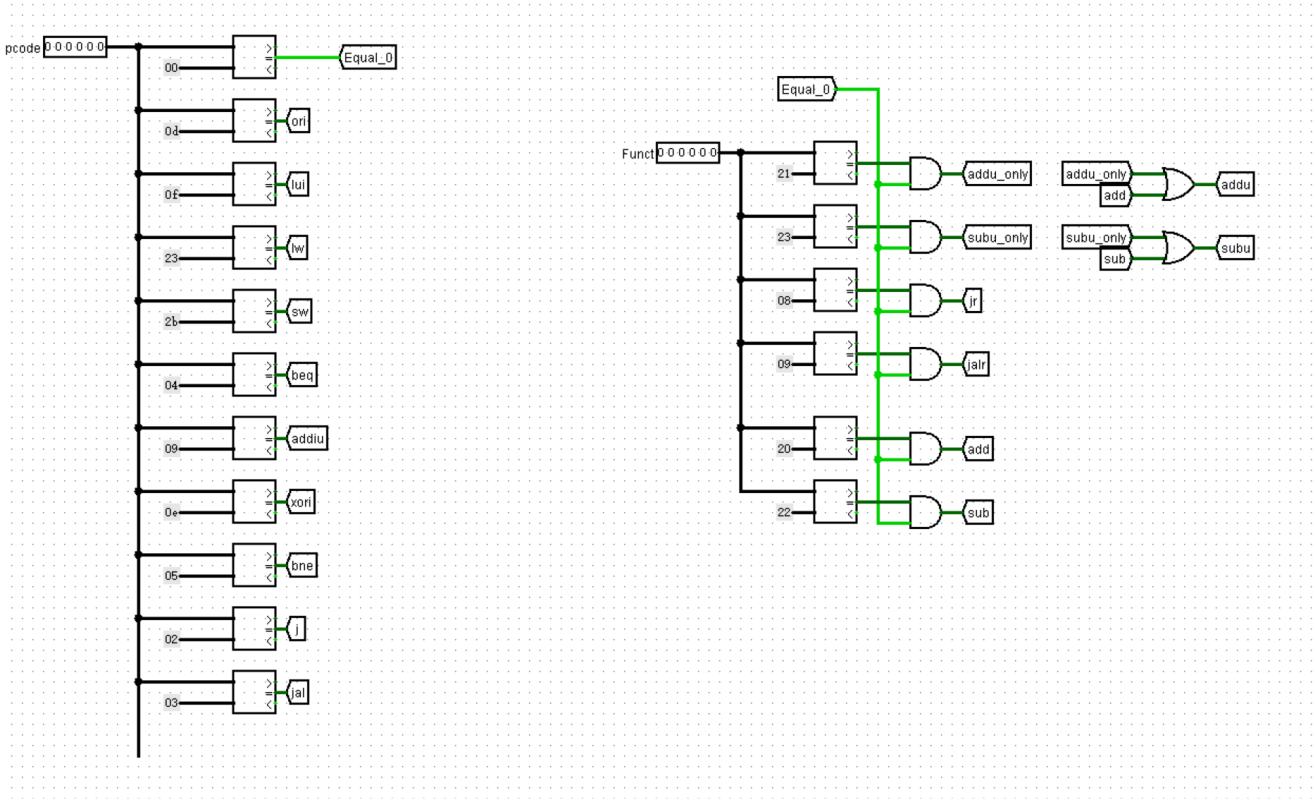
J型地址高4位来自PC4，而不是原PC。在PC=0x0FFFFFFC等进位边界，两者不同。

3.10 PCPlus4：链接写回辅助模块

PCPlus4为32位加法器封装，输入当前PC，输出PC+4。其输出接写回MUX的输入2，供jal/jalr保存返回地址；不改变全局PC信号的含义。

四、Controller 设计

4.1 指令识别与编号



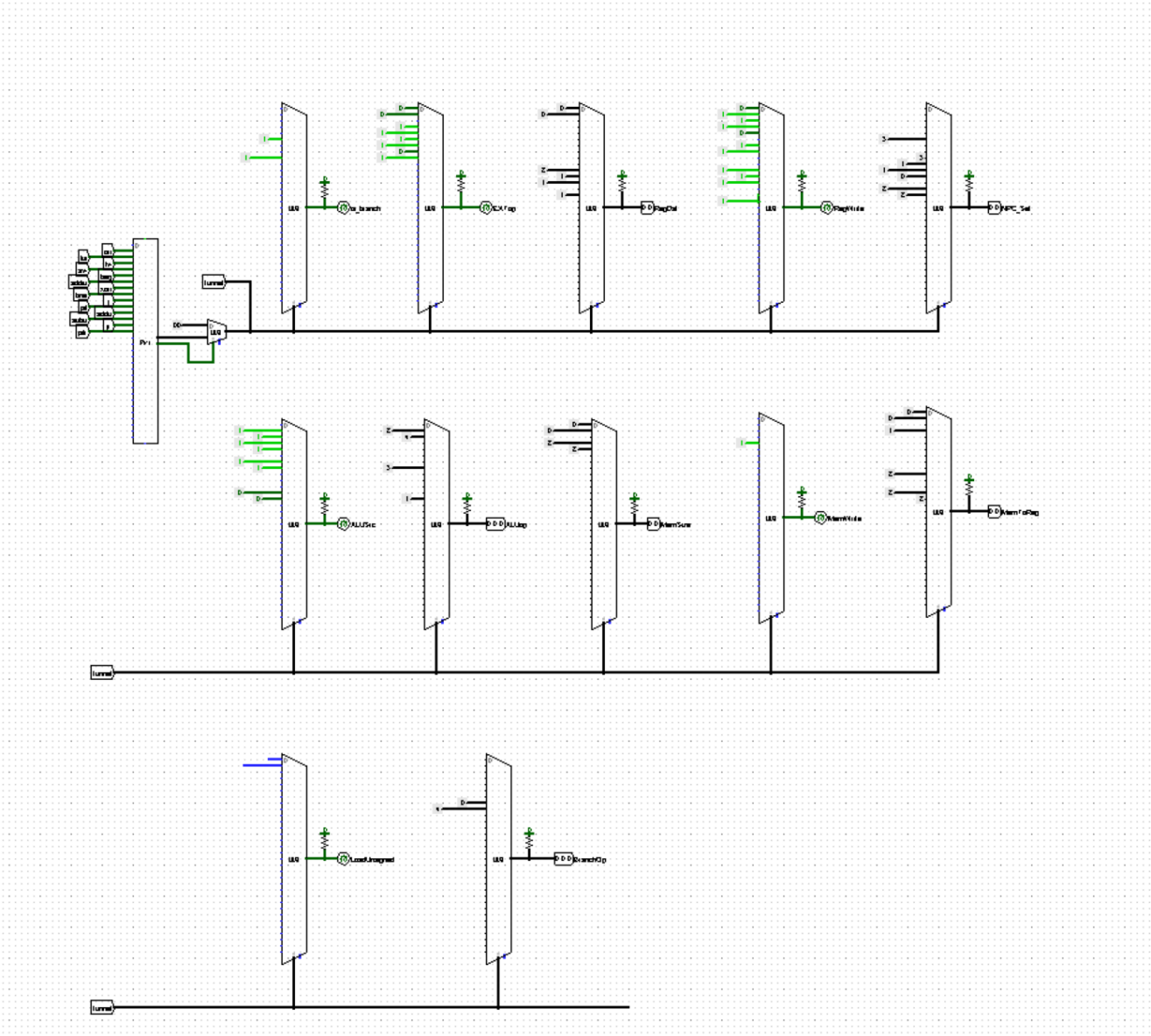
CU输入opcode和Funct，先产生每条指令的识别信号。R型指令要求同时满足opcode=0与相应funct，避免其他指令低6位碰巧相同而被误识别。

识别信号经优先编码器得到5位指令编号。编码器的有效输出配合一个默认选择MUX，使未识别指令使用编号0。编号是工程内部约定，不是opcode，也不是ALUop。

编号	指令	编号	指令
0	默认/nop	8	bne
1	ori	9	j
2	lui	10	jal
3	lw	11	add / addu
4	sw	12	sub / subu
5	beq	13	jr
6	addiu	14	jalr
7	xori	15 ~ 31	未使用

`add` 与 `addu` 分别检查 funct=0x20/0x21，识别结果取或后共用编号11；`sub` 与 `subu` 同理，共用编号12。新增译码没有改变其他指令编号。Controller译码配图已更新，包含新增的add/sub译码及共用控制编号的或门。

4.2 MUX 查表结构



每个控制信号对应一只MUX，选择端统一接指令编号。MUX的第i个数据输入保存第i条指令的控制值。信号位宽由各MUX独立设置，新增控制信号不需要改变其他表的位宽。

各输出使用下拉提供默认0；非零表项由相应位宽的常量驱动。下拉只处理浮空状态，不会覆盖有效的1，也不能修复错误或冲突。保留表项应保证不产生寄存器和内存写操作。

CU不使用ROM，DM写掩码也不使用ROM，工程中仅IM保留ROM。

4.3 控制信号说明

按画布每排从左到右、再从上到下列出：

顺序	信号	位宽	编码与作用
第一排1	is_branch	1	0：非条件分支；1：条件分支，不等于条件已成立
第一排2	EXTop	1	0：立即数零扩展；1：立即数符号扩展
第一排3	RegDst	2	0：rt；1：rd；2：固定31；3保留
第一排4	RegWrite	1	1：上升沿允许GRF写入
第一排5	NPC_Sel	2	0：顺序；1：J型；2：寄存器；3：条件分支
第二排1	ALUSrc	1	0：GRF.RD2；1：EXT输出
第二排2	ALUop	3	0加、1减、2或、3左移、4拼接、5异或
第二排3	MemSize	2	0字节、1半字、2整字、3保留
第二排4	MemWrite	1	1：允许DM写入
第二排5	MemToReg	2	0：ALU结果；1：DM读出；2：PC4；3保留
第三排1	LoadUnsigned	1	0：内存字节/半字符号扩展；1：零扩展
第三排2	BranchOp	3	0：beq；1：bltz/bgez；2：bgtz；3：blez；4：bne

EXTop作用于指令中的立即数，LoadUnsigned作用于从内存读出的数据，两者不应混淆。

4.4 当前指令控制表

表中数值为十进制。这里记录当前电路的实际配置；未参与结果的信号也列出其实际值。

指令	is_branch	EXTop	RegDst	RegWrite	NPC_Sel	ALUSrc	ALUop	MemSize	MemWrite	MemToReg	LoadUnsigned	BranchOp
ori	0	0	0	1	0	1	2	0	0	0	0	0
lui	0	0	0	1	0	1	4	0	0	0	0	0
lw	0	1	0	1	0	1	0	2	0	1	0	0
sw	0	1	0	0	0	1	0	2	1	0	0	0
beq	1	1	0	0	3	0	0	0	0	0	0	0
addiu	0	1	0	1	0	1	0	0	0	0	0	0
xori	0	0	0	1	0	1	5	0	0	0	0	0
bne	1	1	0	0	3	0	0	0	0	0	0	4
j	0	0	0	0	1	0	0	0	0	0	0	0
jal	0	0	2	1	1	0	0	0	0	2	0	0
add / addu	0	0	1	1	0	0	0	0	0	0	0	0
sub / subu	0	0	1	1	0	0	1	0	0	0	0	0
jr	0	0	0	0	2	0	0	0	0	2	0	0
jalr	0	0	1	1	2	0	0	0	0	2	0	0
默认/nop	0	0	0	0	0	0	0	0	0	0	0	0

jr的MemToReg虽然为2，但RegWrite=0，不会写回寄存器。beq/bne的EXTop虽然为1，其分支地址仍由NPC使用独立imm16输入计算。ALUop=0不会妨碍相等比较，因为比较器独立工作。

五、典型指令的数据通路

5.1 寄存器运算：add/sub、addu/subu

GRF根据rs、rt读出两个操作数；ALUSrc=0。ALU完成加减运算，写回MUX选择ALUResult，写地址MUX选择rd。上升沿写入GRF，同时PC更新为PC4。

5.2 立即数运算：ori/addiu/xori/lui

ori/xori先对立即数零扩展，addiu对立即数符号扩展；ALUSrc=1，结果写入rt。lui使用ALU拼接功能，借助rs=0使低16位为0。

5.3 访存：lw/sw

```
GRF.RD1 + SE(imm16) → ALUResult → DM.addr / MemAddr
GRF.RD2                         → DM.WD   / MemData
```

lw设置MemWrite=0、MemSize=2、MemToReg=1，读出的32位数据写入rt。sw设置MemWrite=1、MemSize=2、RegWrite=0，将RD2写入地址对应的整字。

ALUResult是地址，不是要存入内存的数据；DM.WD不能接ALUResult。

5.4 条件分支：beq/bne

两个寄存器读出值通过ALU比较得到Equal。CU设置is_branch=1，BranchOp分别为0/4，BranchControl输出BC_result。NPC_Sel=3；条件成立选择BTarget，否则选择PC4。两条指令均不写GRF、DM。

5.5 跳转与链接：j/jal/jr/jalr

指令	地址来源	是否写GRF	写入目标	写回值
j	JTarget	否	—	—
jal	JTarget	是	31	PC4
jr	GRF.RD1	否	—	—
jalr	GRF.RD1	是	rd	PC4

无条件跳转的is_branch=0，因此BC_result=0；NPC仍能通过输入1/2选择跳转地址。返回地址写回的是PC4，不能直接把当前PC接到链接输入。

六、测试方案与结果

6.1 测试目标与方法

被测文件为 `single_cycle_CPU_final.circ`，以补充 `add/sub` 后的版本为准。测试分为部件检查、基础指令定向测试、随机组合测试和长程序压力测试，最后回归扩展指令。

使用 `checker/logisim-cpu-autotester` 的实际 Logisim 2.7.1 仿真引擎与参考模型逐周期比较。checker仅在内存中向IM注入程序，不改写磁盘上的电路。每个测试从复位开始，保持内置时钟，不通过强制推进PC绕过跳转。

在时钟边沿前观察 `Instr`、`pc`、`RegWrite`、`RegAddr`、`RegData`、`MemWrite`、`MemAddr`、`MemData`；写使能有效时核对对应地址和数据，边沿后另外检查PC。不能要求写使能无效时所有地址、数据都为零。DM通过存取程序和后续写回验证，不直接读取物理RAM。

先用 `ori/lui` 构造不同位模式，再测试运算、存取和分支；用 `sw` 将关键结果写入约定地址，同时保留逐周期日志。这样既能观察最终结果，也能定位首次出错的指令。

6.2 定向测试覆盖方案

对象	需要覆盖的情况	核对重点
add/sub	零、正负位模式、最高位为1、32位回绕、减法下溢、同源同目的寄存器、连续数据依赖	结果保留低32位，写rd，下一地址PC+4
ori	立即数0、0x7FFF、0x8000、0xFFFF，源高16位非零，源与目的相同	立即数零扩展，源高位不会被清除
lui	立即数0、最高位为1、0xFFFF，目的寄存器原值非零	高16位取立即数，低16位必须清零
lw/sw	多地址、重复覆盖、非零基址、正负偏移、首末合法地址、32位数据	地址为基址加符号扩展偏移，按字访问，RD2作为写数据
beq	成立/不成立、前跳/后跳、偏移0、相同寄存器、不同寄存器同值	条件正确，目标为PC+4+(SE(imm16)<<2)
nop	运算、访存、分支之间插入全零指令	不写GRF、不写DM，PC顺序更新
GRF	写入$zero、读取$zero、覆盖非零寄存器、读写相关	$zero恒为0，其他寄存器只在使能时写入
译码隔离	非R型指令的低6位恰好等于某个funct，访存时rs=rt	不能误识别为R型或条件分支
reset	执行后在不推进时钟时拉高reset	PC立即恢复0x3000；另外检查各存储部件的复位连接

6.3 当前版本的实际测试结果

2026-10-31补充 `add/sub` 译码后重新测试：

测试套件	通过/总数	已核对周期	结果文件
基础指令定向 full	147/147	2349	`checker-results/add-sub-full.json`
随机程序 random	100/100	4989	`checker-results/add-sub-random.json`
最高强度 stress	18/18	30149	`checker-results/add-sub-stress.json`
基础测试合计	265/265	37487	三份报告分别保存
原有扩展指令回归	12/12	493	`dm_validation/add_sub_regression_results.json`

基础checker支持 `add`、`sub`、`ori`、`lw`、`sw`、`beq`、`lui`、`nop`；100组随机测试使用固定种子 `20261031`。定向用例包含加法回绕、减法下溢、ori零扩展、lui覆盖、负偏移访存以及分支成立、不成立和后跳等情况。

扩展回归使用独立的整机行为模型，覆盖原有 `addu/subu`、`addiu`、`xori`、`bne`、`j`、`jal`、`jr`、`jalr` 等，12组程序均通过。它与基础checker的参考模型不同，结果单独列出。

顶层接口检查确认只有一个输入Pin `reset`，使用内置Clock；实测PC从 `0x3004` 在不推进时钟、仅拉高reset并传播后恢复到 `0x3000`。IM保持12位地址、32位数据，容量4096字。

七、历史问题修复记录与设计注意事项

以下10项是开发过程中遇到的**历史问题**，均已在 `single_cycle_CPU_final.circ` 中修复，不是当前版本仍存在的故障。2026-10-31重新复查：12组整机程序测试、493个周期全部通过，NPC的840组独立检查全部通过。

历史问题	当时表现	修正及原因	当前状态
新增信号未在顶层接通	CU内部有正确BranchOp，BranchControl输入仍为x	从CU输出端接到同名3位Tunnel，子电路内部正确不等于顶层已经接通	已修复
内存写数据悬空	sw使能有效，但DM.WD为未知值	连接RF_RD2到MemData，并同时送DM和观察端口	已修复
内存宽度误选	lw/sw实际只访问一个字节	MemSize设置为2，即整字	已修复
分支偏移低位拼成11	跳到未对齐的小地址	低位拼00，符号扩展后还要加PC4	已修复
分支结果作为全局总开关	j/jr也被迫要求BC_result=1	分支选择只放在NPC输入3内	已修复
bne的NPC_Sel仍为0	比较结果成立但PC仍顺序更新	bne与beq都选择NPC_Sel=3	已修复
链接输入接当前PC	jal保存了自身地址	改接PCPlus4输出	已修复
jalr控制沿用普通默认值	写到rt/0号寄存器或写回ALU值	RegDst=1选择rd，MemToReg=2选择返回地址	已修复
J型高位取自PC	进位边界跳转错误	高4位改取PC4	已修复
多块ROM用于查控制表	测试加载入口可能产生歧义	CU与DM控制表使用MUX，仅IM使用ROM	已修复

调试时先看PC和Instr，确认取指正确，再看控制信号和寄存器读出值，最后看写回或访存数据。不要只观察最终寄存器值而忽略错误的中间写入。

八、扩展指令时的 RTL 判断表

增加指令时，先读RTL中的数据来源、目标和PC更新方式，再决定控制值；不能只按R/I/J型判断。例如beq和ori都是I型，前者is_branch=1，后者为0。

RTL描述	对应控制判断
R[rt]或R[rd]被更新	RegWrite=1，RegDst分别选0/1
固定写R[31]	RegDst=2
运算第二项来自扩展立即数	ALUSrc=1，并判断EXTop
结果来自内存	MemToReg=1
结果来自返回地址	MemToReg=2
向内存写数据	MemWrite=1；MemSize由写入宽度决定
根据条件更新PC	is_branch=1、NPC_Sel=3，BranchOp选具体条件
直接跳到J型地址或寄存器地址	NPC_Sel分别选1/2，is_branch=0

当前预留能力

- lb/lbu/lh/lhu/sb/sh：DM已提供字节、半字选择与扩展；还需CU译码和表项。注意加载的零扩展不改变地址偏移仍需符号扩展这一事实。
- bltz/bgez/bgtz/blez：BranchControl已有条件逻辑；当前CU只输入opcode和Funct，识别REGIMM类指令时还需考虑rt字段，避免误识别其他编码。
- 移位：ALU有一条左移通路，但要执行标准移位指令，还要核对被移位数、shamt/寄存器移位量的输入选择。
- 乘除法、HI/LO、CP0、异常与中断：当前未实现，需要新增存储状态、数据通路和控制。

九、思考题

9.1 哪些模块存储状态，哪些模块实现状态转移?

可以把单周期CPU看作一个由时钟驱动的大状态机。它的动态状态主要包括PC、通用寄存器和数据存储器的内容。当前状态和取出的指令经过组合逻辑计算，得到下一个时钟沿要写入的状态。

分类	本设计中的模块	说明
动态状态存储	PC	保存当前指令地址，上升沿更新为NPC，异步复位后对外为0x3000
动态状态存储	GRF	保存32个通用寄存器的内容，写使能有效时更新目标寄存器，$zero恒为0
动态状态存储	DM中的RAM	保存程序运行中的数据，存储指令在有效时钟沿改变相应单元
固定程序存储	IM中的ROM	保存程序，也是存储部件；正常执行期间内容不变，不属于被指令逐周期修改的动态状态
状态转移计算	NPC、BranchControl、PCPlus4	计算顺序、分支和跳转地址，决定下一PC及链接返回地址
状态转移计算	Controller、SPLITTER	识别指令、提取字段，决定是否写入、写到哪里以及选择什么数据
状态转移计算	ALU、EXT和各MUX	计算运算结果、有效地址、扩展立即数和写回数据

因此，不是“有一个时钟输入的模块就是状态机”：GRF和DM内部既有保存内容的存储部件，也有组合读出、地址选择等逻辑。分类时应看内部实际功能。

9.2 IM用ROM、DM用RAM、GRF用Register是否合理?

对本次教学CPU而言，这样划分合理，三类存储承担的访问需求不同。

部件	当前实现的理由	限制与改进方向
IM：ROM	程序预先加载，执行期间只读；组合读出便于在同一周期完成取指	无法由CPU执行写指令来修改程序；若需要动态装载或自修改代码，可改用有写入接口的存储器，但要明确装载与执行阶段
DM：RAM	需要读写数据，RAM直接提供地址访问和写使能	要确认读取时序。若改用同步读RAM，读数据通常不能立即用于当前单周期写回，必须重新安排时序或改为多周期/流水线
GRF：Register阵列	容量只有32×32位，需要两个独立读端口和一个写端口；寄存器加MUX、译码器容易实现，并能明确保护$zero	元件和连线较多。可以进一步封装读MUX和写译码；真实硬件可使用合适的多端口寄存器堆实现，不能直接用一块普通单端口RAM替代

本设计将DM组织为四个8位RAM字节通道，按写掩码独立使能，便于后续支持字节、半字写入。这属于存储组织方式的改进，不代表CU已实现所有字节、半字指令。

还应区分器件存储与CPU状态：ROM确实保存信息，但运行中不改变；PC、GRF和可写DM才决定程序执行过程中不断变化的状态。采用指令与数据分离的结构，也避免单端口存储器同时取指与访存的冲突。

9.3 是否额外设计了模块？设计思路是什么？

除了常见的PC、IM、GRF、ALU、DM和控制器，本设计还将一些功能独立封装：

模块	接口与功能	独立封装的原因
SPLITTER	输入32位Instr，输出opcode、rs、rt、rd、shamt、funct、imm16、imm26	指令位段只在一个位置定义，减少到处手动拆线产生的错误
BranchControl	输入分支使能、BranchOp、寄存器值、Equal和Judge，输出分支成立结果	把不同分支条件的判断与NPC地址计算分离，增加条件分支时职责清楚
PCPlus4	输入PC，输出PC+4，供链接写回MUX选择	避免jal/jalr误把当前PC作为返回地址；当前电路仍保留NPC内部的PC+4加法器，后续可统一提供以减少重复
EXT	输入16位立即数及EXTop，输出零扩展或符号扩展结果	将立即数解释与ALU运算解耦；各指令通过控制信号选择扩展方式

这些模块应表达明确的功能，而不是直接识别所有指令。例如BranchControl负责“条件是否成立”，NPC负责“目标地址是多少、选哪条路径”。分支结果只控制NPC的分支候选输入，不能作为所有跳转的总开关，否则j/jr也会被分支条件阻挡。

9.4 为什么nop可以不加入控制信号真值表？

本题规定 `nop=0x00000000`。它不会命中目前有效指令的译码，默认路径令 `RegWrite=0`、`MemWrite=0`、`is_branch=0`、`NPC_Sel=0`。于是GRF和DM不更新，PC正常加4，已经满足nop语义，无需为它另设一个有效指令编号。

这里的关键是**默认控制必须安全**，不是“机器码全零就一定没有动作”。如果默认路径允许写入或错误跳转，nop仍会出错。ALU和组合读端口可以继续产生结果，只要没有错误地提交到存储状态即可。

本设计即使对其他运算允许写入0号寄存器，GRF也会屏蔽写入；但nop本身的控制路径仍应关闭RegWrite，不能只依赖$zero保护。PC加4是正常取下一条指令，不违背“空指令不修改有效数据”的要求。

9.5 如何评价MIPS单元6.1给出的测试样例?

以下评价依据课程样例的完整代码：先用ori、lui构造数据，执行三条add，再将数据写入地址0～24；随后lw并转存到28、32；最后两条beq通过地址36、40的写入情况观察结果。样例明确暂不考虑延迟槽。

优点。 测试按照依赖顺序展开：先验证容易直接观察的ori/lui，再用其构造正数和负数测试add，之后先sw再lw，最后测试beq。这样能避免寄存器和内存全为零时，错误实现也碰巧读出零。将关键结果转存到内存，也便于与Mars比较。连续使用上一条指令的结果，还能初步检查寄存器写入后下一周期能否读到新值。

但它更适合“先跑通数据通路”，不足以作为完整验收：

对象	样例已经覆盖	具体不足与补充方法
指令集合	ori、lui、add、sw、lw、beq	对样例假定的六条指令都有涉及；相对本次八条要求，缺少sub和nop，需另补减法、空操作和PC顺序更新测试
ori	零源寄存器、非零源寄存器，且有连续依赖	123、456都没有设置立即数第15位，无法暴露错误符号扩展；补0x8000/0xFFFF以及源高16位非零的情况
lui	123和0xFFFF，覆盖立即数最高位0/1	两个目标寄存器起初为0，不能有效检查“清除旧低16位”；先向目标写入低位非零值，再执行lui
add	正正、正负、负负，结果含非零与负数位模式	缺少0参与、跨最高位边界、32位回绕、写$zero、rd与rs/rt重合等；题目不检查溢出，仍应核对低32位
sub	无	补相等相减、0减1、正负位模式、结果为0/负数以及目的寄存器重合
sw/lw	多个按字对齐地址；存入后读出，并将读出值再次写入	基址只有0，偏移均为非负数；补非零基址、负偏移、同地址覆盖、首末合法地址以及lw目的寄存器与基址寄存器相同
beq	第一次不相等不跳，第二次相等跳，均为前向分支	没有后向分支、负偏移和偏移0；相等使用了同值但不同编号的寄存器，还应补同一寄存器与$zero等情况
nop与复位	未专门验证	加入全零指令，确认两个写使能为0且PC加4；在已有非零状态后拉高reset检查复位
检查方式	最终查看内存并与Mars比较	不同错误可能互相抵消，或错误中间写入被覆盖；补逐周期PC、译码和写回检查，并保存首次不匹配日志

两条beq的实际路径值得单独分析：

```
beq $a0, $a1, loop1     # 1 != 2，期望不跳；目标偏移为+1
beq $a0, $a2, loop2     # 1 == 1，期望跳转；目标偏移也为+1
loop1: sw $a0, 36($t0)
loop2: sw $a1, 40($t0)
```

正常执行时跳过地址36的存储，最终 `M[36]=0`（此前未写入且内存已清零），`M[40]=2`。如果第一条错误地跳转，会执行loop1；如果第二条错误地不跳，也会执行loop1。因此这段程序能够暴露常见的条件错误，却仅凭最终 `M[36]=1` 不容易区分是哪一条分支出错。

另外，两次目标偏移都恰好是+1，无法充分区分某些错误的地址计算。例如把 `PC+4+(imm<<2)` 错接成 `PC+(imm<<2)`，第二条分支会落到loop1；但要定位移位、符号扩展和基准地址各自的错误，仍应增加不同正偏移、负偏移循环和偏移0测试。

因此，应保留该样例作为入门冒烟测试，再按6.2节补充边界、控制无效时的副作用和组合程序测试。编译前还要检查Mars实际生成的基本指令，避免伪指令或超范围立即数被自动展开，导致测试超出CPU支持范围。

十、参考资料与版本说明

本文参考以下范例的“总体设计—模块接口—功能定义—数据通路—测试”组织方式；接口名称、控制编码和存储规模均以本工程为准，不直接沿用范例中的控制值。

1. FlyingLandlord：P3 课下学习——单周期 CPU 设计
2. theUHO：P3-CPU设计文档
3. Hyggge：P3 单周期 CPU（Logisim实现）：BranchControl的模块划分和电路布局参考。

配图来自本次提供的实际截图，放在 `images/设计文档副本/`。顶层图由两张截图对齐去重拼接；其余部件截图保留原图。IM、Controller译码与控制表配图已替换为最新提供的截图；控制表图包含第三排的LoadUnsigned和BranchOp。具体控制值以本文表格及电路文件为准。

后续修改电路时，应同步更新指令支持范围、端口位宽、MUX输入编号和控制表，并重新记录对应版本的测试结果。